UTAA - Faculty of Engineering

CENG 454: Game Programming
Spring Term 2025-2026

Project: Sky-High Prototype

PDF Flight Log Report

Full Name	Egemen Caliskan
Student ID	220444035
GitHub URL	https://github.com/Jeithx/CENG454-HW1-Egemen-Caliskan-220444035
Submission Date	22 March 2026

Section 1: Screenshots

Screenshot 1: GitHub Student Developer Pack - Education Benefits

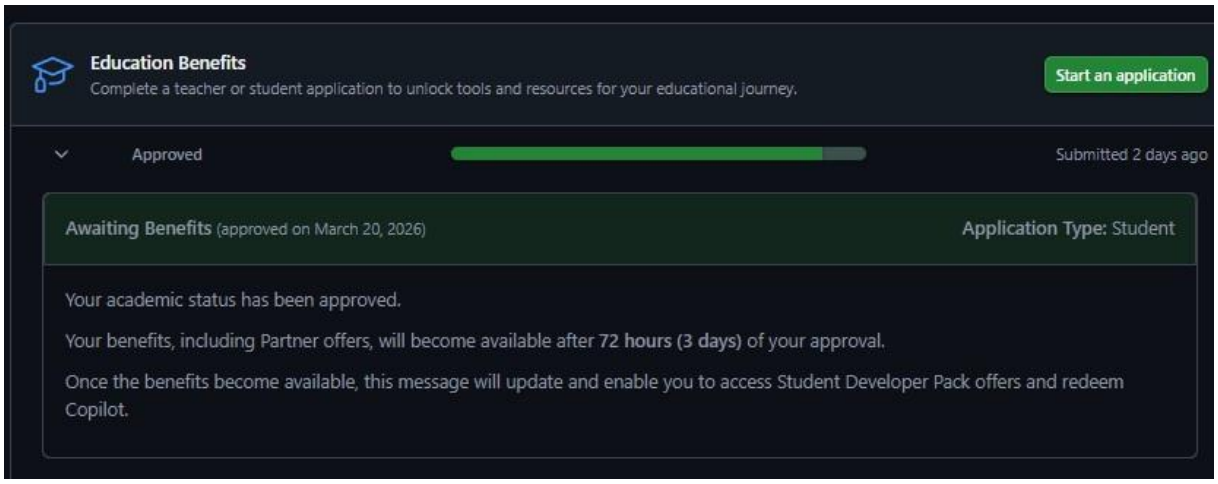


Figure 1: GitHub Education Benefits - Academic status approved (March 20, 2026)

Screenshot 2: Unity Project Settings - Force Text Serialization + Visible Meta Files

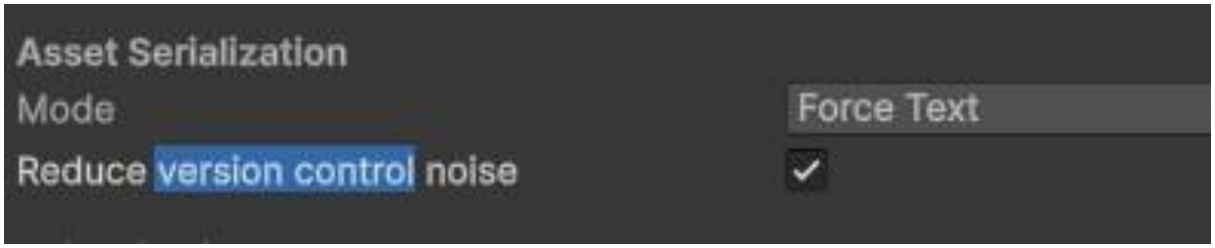


Figure 2a: Asset Serialization Mode set to Force Text

Figure 2b: Version Control Mode set to Visible Meta Files

Screenshot 3: GitHub Project Kanban Board - All 3 User Story Cards in Done

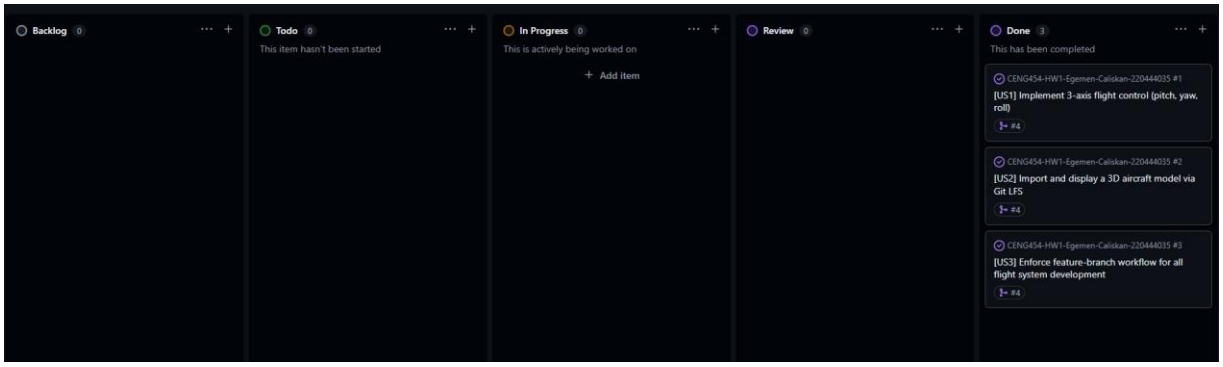
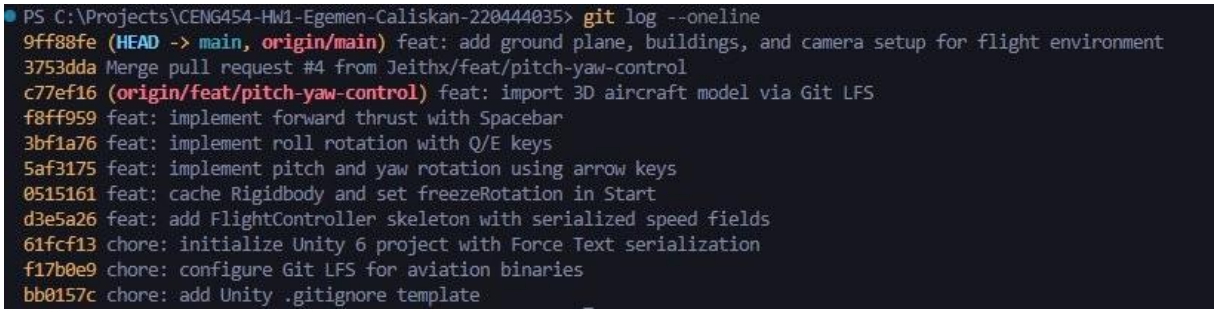


Figure 3: Kanban board with all three User Story cards moved to Done column

Screenshot 4: Terminal - git log --oneline (Atomic Commits)



```
PS C:\Projects\CENG454-HW1-Egemen-Caliskan-220444035> git log --oneline
9ff88fe (HEAD -> main, origin/main) feat: add ground plane, buildings, and camera setup for flight environment
3753dda Merge pull request #4 from Jeithx/feat/pitch-yaw-control
c77ef16 (origin/feat/pitch-yaw-control) feat: import 3D aircraft model via Git LFS
f8ff959 feat: implement forward thrust with Spacebar
3bf1a76 feat: implement roll rotation with Q/E keys
5af3175 feat: implement pitch and yaw rotation using arrow keys
0515161 feat: cache Rigidbody and set freezeRotation in Start
d3e5a26 feat: add FlightController skeleton with serialized speed fields
61fcf13 chore: initialize Unity 6 project with Force Text serialization
f17b0e9 chore: configure Git LFS for aviation binaries
bb0157c chore: add Unity .gitignore template
```

Figure 4: git log --oneline showing 10 atomic commits with the environment update

Screenshot 5: Terminal - git lfs ls-files

```
PS C:\Projects\CENG454-HW1-Egemen-Caliskan-220444035> git lfs ls-files
6f20db6610 * Assets/Civil Transport Aircraft/Textures/YellowPalette.png
61a7e28e0b * Assets/Models/Aircraft/Transporter_Airplane_1.fbx
1d17a9ff35 * Assets/TutorialInfo/Icons/URP.png
PS C:\Projects\CENG454-HW1-Egemen-Caliskan-220444035>
```

Figure 5: git lfs ls-files confirming .fbx aircraft model is LFS-tracked

Screenshot 6: Unity Game View - Aircraft Model with Environment

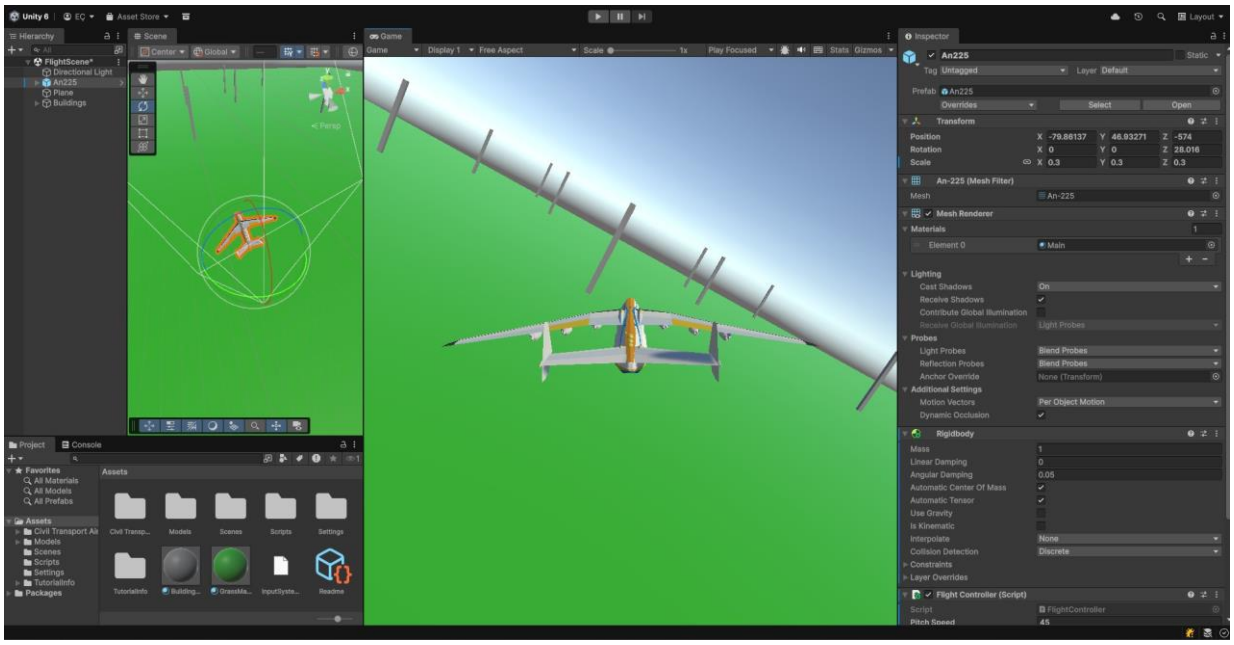


Figure 6: Aircraft rendered in Game View with ground plane, buildings, Rigidbody and FlightController attached. No pink textures.

Screenshot 7: GitHub Pull Request - Merged with Aero-Review Checklist

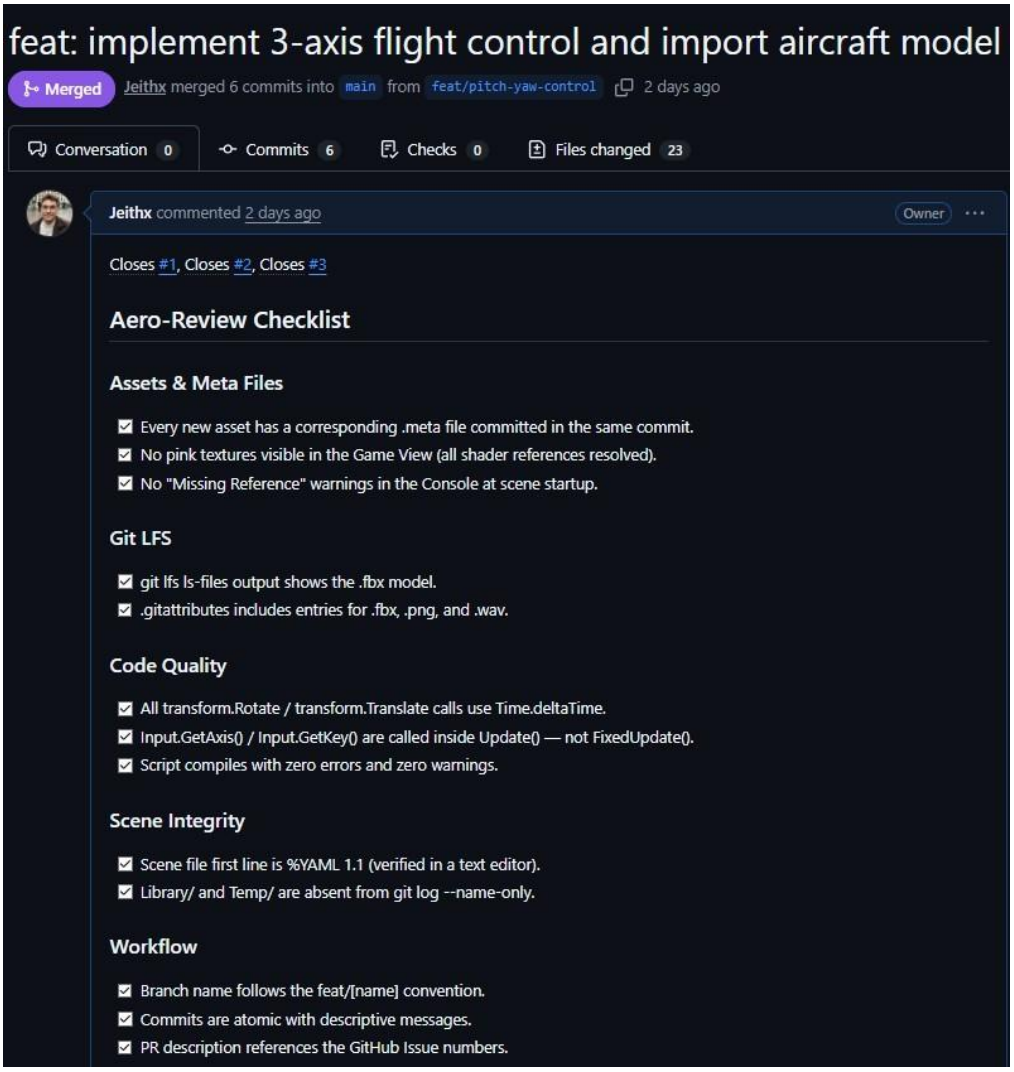


Figure 7: PR merged into main with complete Aero-Review Checklist verified

Section 2: Code Analysis

(a) Why Time.deltaTime Is Mandatory

I ran into this problem firsthand. Built the project, sent it over to a friend, and on his old laptop the plane was barely crawling. On my desktop? Totally fine, nice and smooth. I was confused for a good while. Then it clicked — Update() just runs once per frame, that's it. My desktop does like 144 frames a second so the plane gets 144 tiny rotation nudges. His laptop does maybe 35. Way fewer nudges, way slower movement. Same code, completely different behavior.

deltaTime is literally just a number telling you "this many seconds passed since the last frame." On a fast machine that number is tiny, like 0.005. On a slow one it's bigger, like 0.033. When you multiply your speed value by deltaTime, you end up getting the same total rotation per second.

(b) Why rb.freezeRotation = true Is Set

This confused me at first honestly. Why would I slap a Rigidbody on there and then tell it not to rotate? Feels backwards. Without Rigidbody, OnCollisionEnter and similar callbacks won't work, and at some point we'd want the plane to actually bump into things. The problem is our FlightController is already doing transform.

The Rigidbody doesn't care about that though. It's got its own angular velocity thing going on and it tries to mess with the rotation too. I tested it once without freezeRotation and the plane was doing this creepy wobble with no keys pressed. Looked broken. Both systems were fighting over who controls the rotation and kept overwriting each other. freezeRotation = true just makes wobbling gone.

(c) Why Update() Instead of FixedUpdate() for Input

FixedUpdate has its own timer — ticks roughly every 20ms, doesn't care about rendering at all. Sometimes two rendered frames go by between FixedUpdate calls. Sometimes none. A player taps a key right when FixedUpdate isn't running? That input just disappears. Nobody picks it up. Gone. Update runs every single rendered frame though. Catches everything, never misses a keypress.

Now if we were doing AddForce or AddTorque for a real physics flight model, we'd need FixedUpdate for that. Force calculations get messed up if the time between calls keeps jumping around — you get jittery, inconsistent movement.

Section 3: Task 1 Analysis Questions

Why does Unity need Force Text serialization for team version control?

Unity can save scene files in binary or as YAML text. Some project templates even default to binary or mixed mode which is annoying. The problem with binary is that Git can't do anything useful with it. You run git diff and all you see is "binary files differ" — completely useless. You can't tell if someone moved a light or deleted half the scene.

Merging is even worse. Git's whole merge thing is about comparing lines of text. A binary blob doesn't have "lines" so Git just gives up and says pick one, the other is gone. With Force Text turned on, Unity writes YAML instead. Git can read YAML just fine — you see exactly which properties changed, three-way merges actually work (not perfectly, YAML scenes still get messy sometimes, but at least it's possible). Running a team project without Force Text is asking for trouble. I know because I tried it once on a smaller project and had to redo an entire scene from scratch.

What is the purpose of a .meta file? What happens if you commit an asset without its .meta?

Every file you drop into Assets gets a .meta file next to it. The important thing inside is a GUID — like a fingerprint Unity stamps on each asset. Here's the key thing: when a scene uses a texture, it doesn't store the file path anywhere. It stores the GUID. Prefabs, scenes, materials... basically everything in Unity talks to each other through these GUIDs instead of file paths.

So if you commit a texture but forget the .meta? Your teammate pulls, opens Unity, and the engine goes "this file has no GUID" and makes up a new one. Different from yours. Now every material that was pointing at the old GUID shows up magenta pink. Broken references everywhere. The .meta also keeps import settings in it (texture compression, model scale, that kind of thing) so those get lost too. So asset and .meta should go into repo in the same commit.

Section 4: Sprint Retrospective

Writing the FlightController code was the easy part honestly. Read input, rotate, translate, done. What actually took my time was all the Git stuff around it. I've used Git before sure, but usually it's just "git add, commit, push" on main and call it a day. This time I had to think about branch names, atomic commits, LFS tracking order, making sure .meta files tagged along with every asset. There was this moment early on where I almost imported the FBX before committing .gitattributes. That would've been a nightmare to undo without rewriting history. Caught it just in time though.

Another thing that surprised me was how much time the Kanban board and issue tracking ate compared to actual coding. Writing acceptance criteria for each user story felt pointless at first, but looking back it did help me not forget small details. If I had to redo this, I'd set up all the infra on day one — Git, LFS, .gitignore, Force Text — and only then open Unity. And I'd commit after every tiny change. This whole process made me understand why game studios are so strict about version control. One bad commit can really mess things up for the whole team.

Section 5: Troubleshooting - Flight Report #001

The Binary Scene Incident

So the student saved the scene without turning on Force Text, meaning Unity wrote the .unity file as binary data. Now when someone tries to run git diff on it, Git just goes "nope, binary files differ" and gives you nothing. No line-by-line comparison, no context, just a dead end. And if two people both edited that scene on separate branches? Forget about merging. Git needs text lines to do a merge. Binary doesn't have that. So you pick version A or version B, and everything from the other side is just... gone.

Force Text would've prevented all of this. It makes Unity save scenes as YAML, which is just plain text with key-value pairs. Git can diff YAML line by line, show you proper context. Merges aren't perfect (YAML scene files can still be messy to resolve by hand) but you can at least see what changed and make a decision.

For the fix: go into Unity, Edit > Project Settings > Editor, flip Asset Serialization to Force Text. Then — and this is the part people miss — you have to actually re-save every scene and prefab. Just flipping the setting doesn't convert files that are already binary. Open each scene, Ctrl+S, done. After that, stage everything, commit with something like "fix: re-serialize all scenes as YAML text", push. The old binary commits stay in Git history (you'd need BFG Repo Cleaner or filter-branch to scrub those out, which is usually more hassle than it's worth) but from that point on, diffs and merges work fine.